

데이터 과학 및 분석

보강#1 – Python 기초

호서대학교 조학수



01 Python

02 Numpy

03 Pandas

04 matplotlib

05 seaborn / plotly

01 Python



1 연산자

■ 산술 연산자의 종류

표 4-1 산술 연산자의 종류

연산자	의미	사용 예	설명
=	대입 연산자	$a = 3$	정수 3을 a에 대입
+	더하기	$a = 5 + 3$	5와 3을 더한 값을 a에 대입
-	빼기	$a = 5 - 3$	5에서 3을 뺀 값을 a에 대입
*	곱하기	$a = 5 * 3$	5와 3을 곱한 값을 a에 대입
/	나누기	$a = 5 / 3$	5를 3으로 나눈 값을 a에 대입
//	나누기(몫)	$a = 5 // 3$	5를 3으로 나눈 후 소수점을 버리고 값을 a에 대입
%	나머지값	$a = 5 \% 3$	5를 3으로 나눈 후 나머지값을 a에 대입
**	제곱	$a = 5 ** 3$	5의 3제곱을 a에 대입



1 연산자

■ 산술 연산자의 우선순위

```
a, b, c = 2, 3, 4
print(a + b - c, a + b * c, a * b / c)
```

출력 결과

1 14 1.5

1. 지수 (******)
2. 단항연산자(**+** , **-** , **~**) -5 , ~a
3. 곱셈, 나눗셈, 나머지, 몫(***** , **/** , **%** , **//**)
4. 덧셈, 뺄셈 (**+** , **-**) 3+4
5. 비트 연산자 (**<<** , **>>**) >> 비트 AND (**&**) >> 비트 XOR (**^**) >> 비트 OR (**|**)
6. 비교 연산자 (**==** , **!=** , **>** , **<** , **>=** , **<=** , **is** , **is not** , **in** , **not in**)
7. 논리 not >> 논리 and >> 논리 or
8. 할당 연산자 (**=** , **+=** , **-=** , ***=** , **/=** , **%=** , **//=** , ****=** , **&=** , **!=** , **^=** , **<<=**)
9. 식별 연산자 lambda (lambda x: x+2)

1 연산자

■ 산술 연산자의 우선순위

$$\textcircled{1} (a + b) * c \rightarrow (2 + 3) * 4 \rightarrow 5 * 4 \rightarrow 20$$

$$\textcircled{2} a + (b * c) \rightarrow 2 + (3 * 4) \rightarrow 2 + 12 \rightarrow 14$$

1. 단항연산자($+$, $-$, $\sim$) -5 , $\sim a$
2. 곱셈, 나눗셈, 나머지, 몫($*$, $/$, $\%$, $//$)
3. 덧셈, 뺄셈 ($+$, $-$) $3+4$
 - 산술 연산자는 괄호가 가장 우선
 - 동일 우선순위는 왼쪽에서 오른쪽으로

1 연산자

■ 산술 연산을 하는 문자열과 숫자의 상호 변환

- 숫자 변환 : `int()`, `float()`, `complex()`
- 문자열 변환 : `str()`, `repr()`

[illegible]

출력 결과

101 101.123 10000000000000000000000000000000

1 연산자

■ 산술 연산을 하는 문자열과 숫자의 상호 변환

- 숫자를 문자열로 변환 : `str()`
- 문자열의 `+` 는 문자열의 병합(concatenate)
- `str(a)*3` : 문자열을 3회 반복 병합

```
a = 100; b = 100.123
str(a) + '1'; str(b) + '1'
```

출력 결과

```
'1001'
'100.1231'
```



1 연산자

■ 산술 연산자, 대입 연산자

- 대입 연산자 : =
- 산술 대입연산자 : +=, -=, *=, /=, %=, //=, **=
 - $a+=3$: a에 3을 더해서 다시 a에 대입, $a=a+3$ 과 동일

표 4-2 대입 연산자의 종류

연산자	사용 예	설명
+=	$a+=3$	$a=a+3$ 과 동일
-=	$a-=3$	$a=a-3$ 과 동일
=	$a=3$	$a=a*3$ 과 동일
/=	$a/=3$	$a=a/3$ 과 동일
//=	$a//=3$	$a=a//3$ 과 동일
%=	$a\%=3$	$a=a\%3$ 과 동일
=	$a=3$	$a=a**3$ 과 동일



1 연산자

■ 관계연산자

- 어떤 것이 크거나 작거나 같은 지 비교
 - 참은 True, 거짓은 False
 - 조건문(if), 반복문(while)에 사용, 단독으로는 거의 사용하지 않음

$a < b = \begin{cases} \text{참} & : \text{True} \\ \text{거짓} & : \text{False} \end{cases}$

그림 4-1 관계 연산자의 기본 개념

표 4-3 관계 연산자의 종류

연산자	의미	설명
==	같다.	두 값이 동일하면 참
!=	같지 않다.	두 값이 다르면 참
>	크다.	왼쪽이 크면 참
<	작다.	왼쪽이 작으면 참
>=	크거나 같다.	왼쪽이 크거나 같으면 참
<=	작거나 같다.	왼쪽이 작거나 같으면 참



1 연산자

■ 논리 연산자

- and(그리고), or(또는), not(부정)
 - a가 100과 200 사이에 있다는 조건 표현

$(a > 100) \text{ and } (a < 200)$

표 4-4 논리 연산자의 종류

연산자	의미	설명	사용 예
and(논리곱)	~이고, 그리고	둘 다 참이어야 참	$(a > 100) \text{ and } (a < 200)$
or(논리합)	~이거나, 또는	둘 중 하나만 참이어도 참	$(a == 100) \text{ or } (a == 200)$
not(논리부정)	~아니다, 부정	참이면 거짓, 거짓이면 참	$\text{not}(a < 100)$

1 연산자

■ 비트 연산자

- 정수를 2진수로 변환한 후 각 자리의 비트끼리 연산 수행
- 비트 연산자의 종류 : &, |, ^, ~, <<, >>

표 4-5 비트 연산자의 종류

연산자	의미	설명
&	비트 논리곱(and)	둘 다 1이면 1
	비트 논리합(or)	둘 중 하나만 1이면 1
^	비트 논리적 배타합(xor)	둘이 같으면 0, 다르면 1
~	비트 부정	1은 0으로, 0은 1로 변경
<<	비트 이동(왼쪽)	비트를 왼쪽으로 시프트(Shift)
>>	비트 이동(오른쪽)	비트를 오른쪽으로 시프트(Shift)

1 연산자

■ 비트 논리곱과 비트 논리합 연산자

- and는 그 결과가 참(True) 또는 거짓(False) 논리 값을 도출
- &는 비트 논리곱을 도출
- 비트 연산은 0과 1밖에 없으므로 0은 False, 1은 True

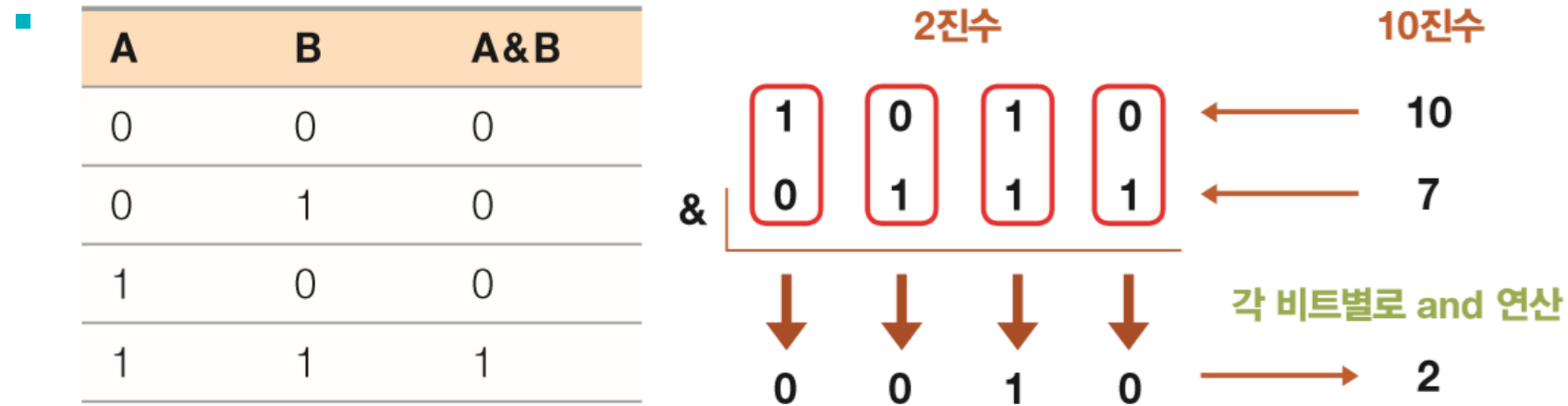


그림 4-2 비트 논리곱의 예

1 연산자

- 123 & 456
 - 123의 2진수인 $111\ 1011_{(2)}$ 와 456의 2진수인 $1\ 1100\ 1000_{(2)}$ 의 비트 논리곱(&) 결과인 $100\ 1000_{(2)}$ 가 됨 10진수로는 72
- 자릿수가 다르면 빈 자리에 0을 채운 후 비트 논리곱 연산 ($1\ \&\ 0xFF = 1$)
- 0과 비트 논리곱을 수행하면 0이 됨

10 & 7

123 & 456

0xFFFF & 0x0000

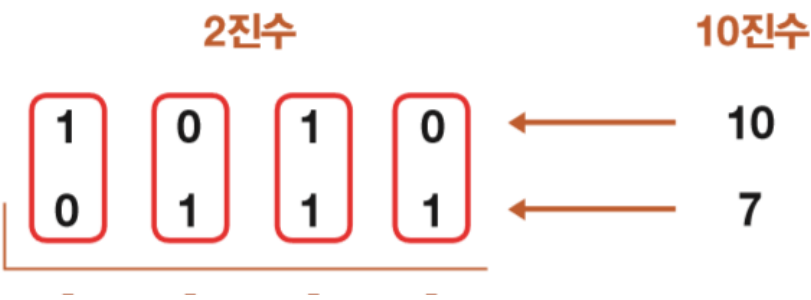
출력 결과

2 72 0



1 연산자

- 비트 논리합(|) : 두 값중 하나라도 1이면 1, 모두 0이면 0



- 1 1 1 1 → 15

1 연산자

- 논리합 연산
 - 10 | 7과 123 | 456
 - 0xFFFF | 0x0000
 - 1 | 0xFF

10 | 7

123 | 456

0xFFFF | 0x0000

출력 결과

15 507 65535

1 연산자

- 비트 배타적 논리합($\wedge$) : 두 값이 다르면 1, 같으면 0

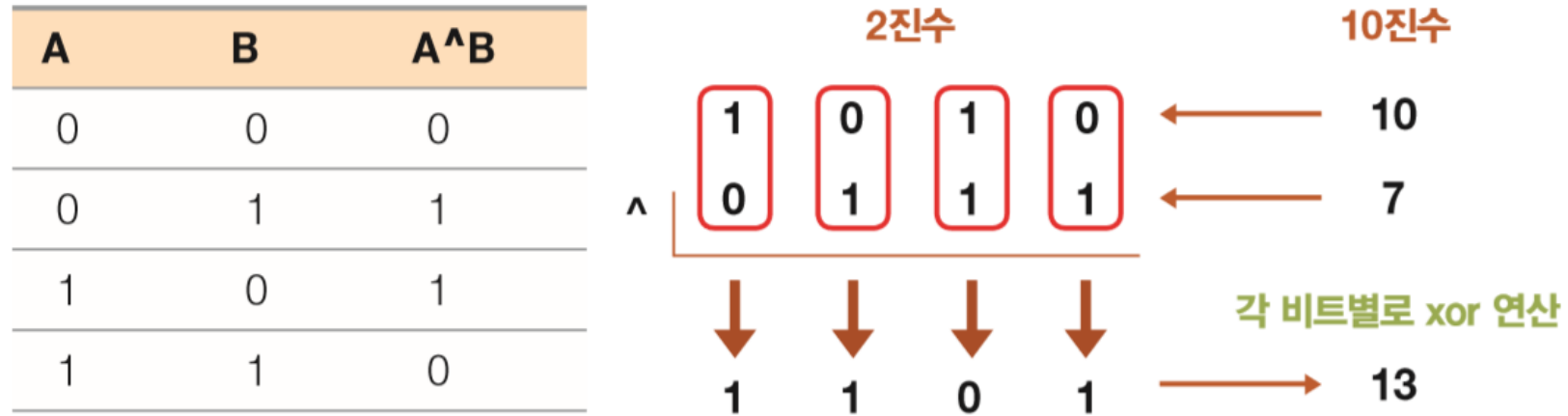


그림 4-4 비트 배타적 논리합의 예

10 ^ 7
123 ^ 456
0xFFFF ^ 0x0000

출력 결과

13 435 65535

1 연산자

■ 시프트 연산자

- 왼쪽 시프트 연산자 : 왼쪽으로 시프트는 2를 곱한 효과

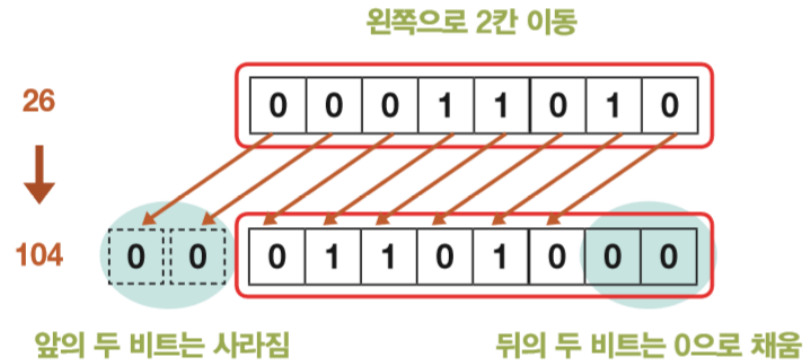


그림 4-7 26의 왼쪽으로 2칸 시프트 연산

`a = 10`

`a << 1; a << 2; a << 3; a << 4`

출력 결과

20 40 80 160

1 연산자

- 오른쪽 시프트 연산자 ($\gg$) : 2로 나눈 몫



그림 4-8 26의 오른쪽으로 2칸 시프트 연산

`a = 10`

`a >> 1; a >> 2; a >> 3; a >> 4`

출력 결과

5 2 1 0

1 연산자

- 연산자 우선순위: 여러 개의 연산자가 있을 경우 정해진 순서

표 4-6 연산자 우선순위

우선순위	연산자	의미
1	() [] {}	괄호, 리스트, 딕셔너리, 세트 등
2	**	지수
3	+ - ~	단항 연산자
4	* / % //	산술 연산자
5	+ -	
6	<< >>	비트 시프트 연산자
7	&	비트 논리곱
8	^	비트 배타적 논리합
9		비트 논리합
10	<> >= <=	관계 연산자
11	== !=	동등 연산자
12	= %= /= //= -= += *= **=	대입 연산자
13	not	논리 연산자
14	and	
15	or	
16	if ~ else	비교식



2 제어문

■ if문

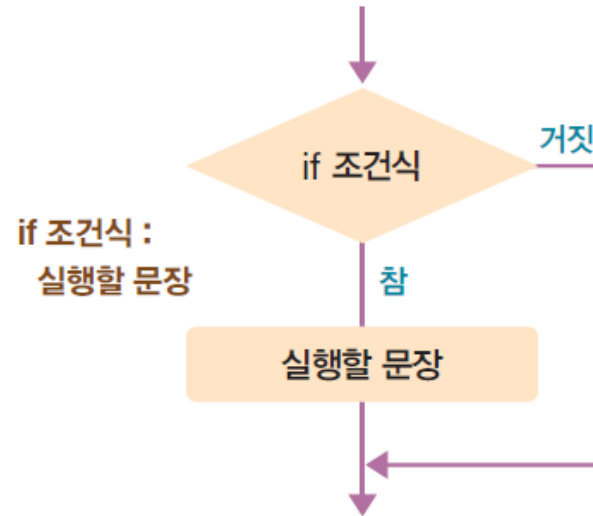


그림 5-1 if 문의 형식과 순서도

```

a = 99
if a < 100 :
    print("100보다 작군요.")
  
```

출력 결과

100보다 작군요.

2 제어문

■ if문

```
a = 99
if a < 100 :
    print("100보다 작군요.")
```

출력 결과

100보다 작군요.

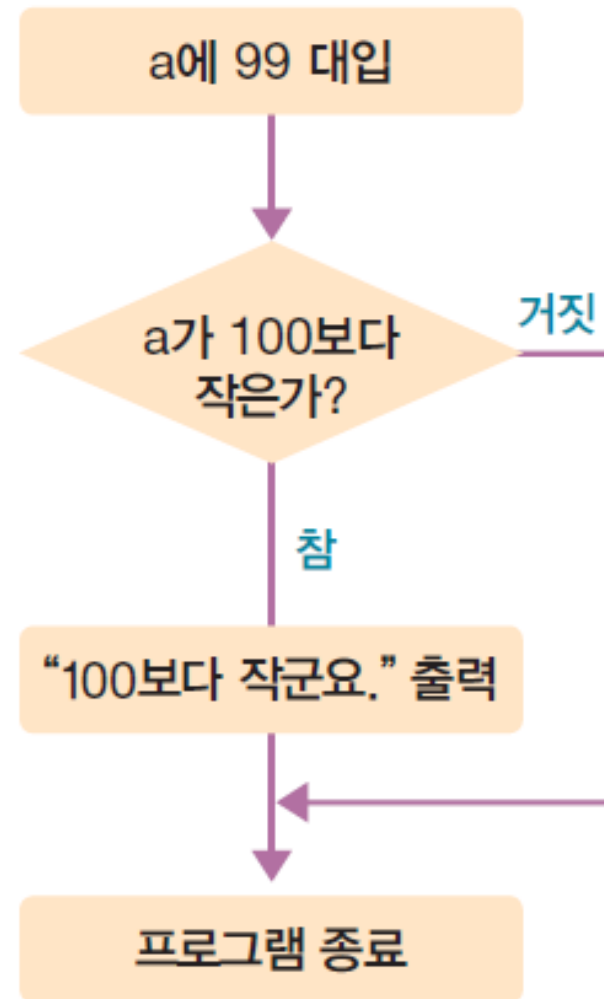


그림 5-2 if 문 실행 과정

2 제어문

■ if~else 문

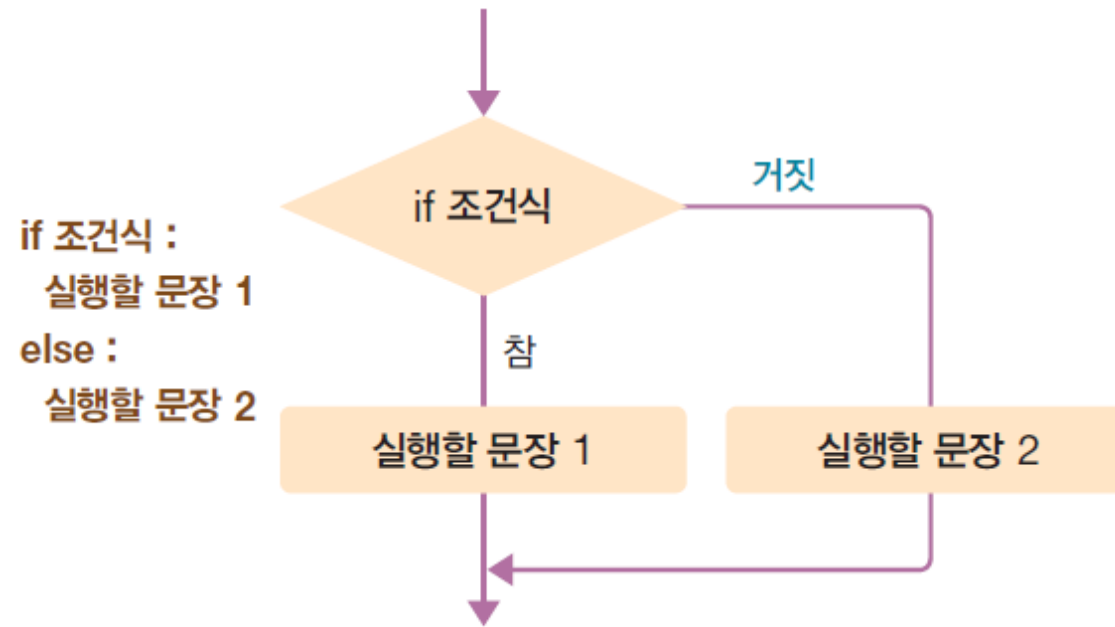


그림 5-3 if~else 문의 형식과 순서도

- 조건이 참일 때와 거짓일 때 실행할 문장이 다름

2 제어문

■ if~else 문

Code05-03.py

```
1 a = 200
2
3 if a < 100 :
4     print("100보다 작군요.")
5 else :
6     print("100보다 크군요.")
```

출력 결과

100보다 크군요.

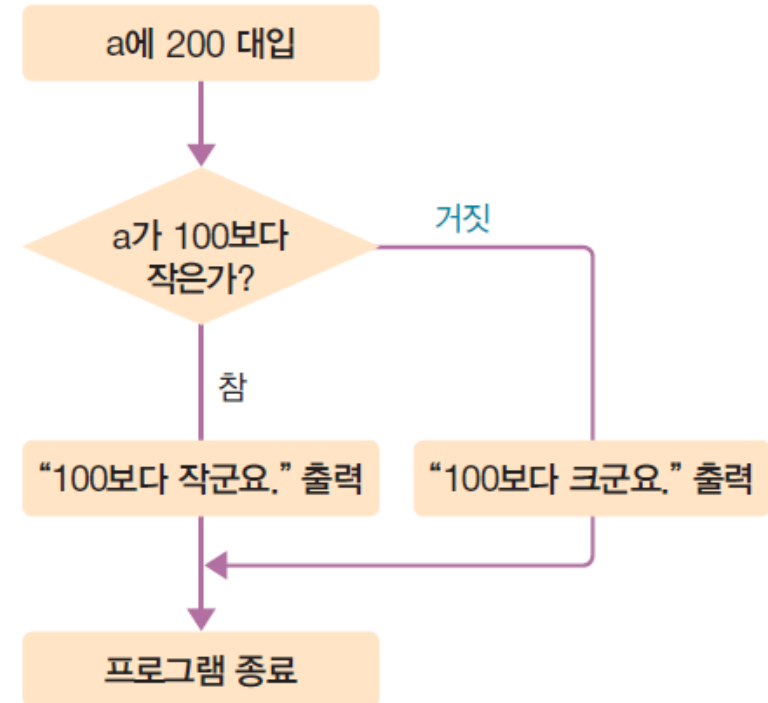


그림 5-4 Code05-03.py 실행 과정



2 제어문

■ 내장 if~else~문 (if~else~if~else 문)

- if 문을 한 번 실행한 후 그 결과에서 if 문을 다시 실행하는 것

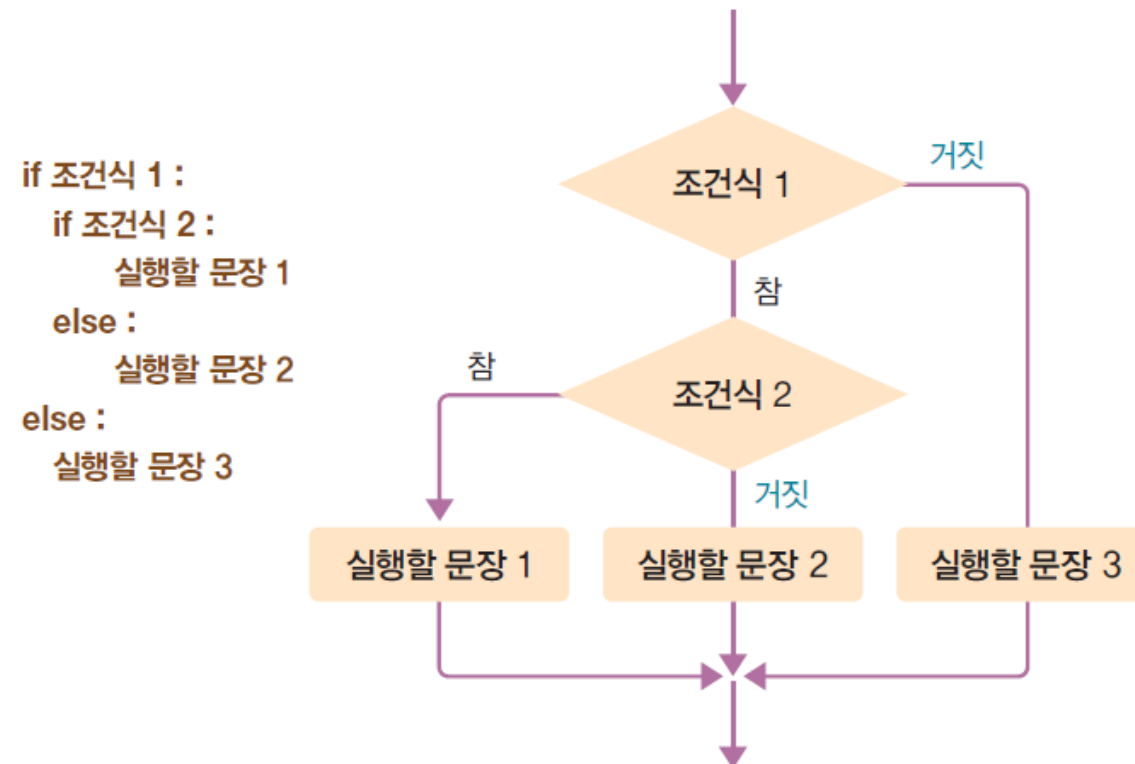


그림 5-5 중첩 if 문의 형식과 순서도

2 제어문

■ 내장 if~else~문 (if~else~if~else 문)

Code05-06.py

```
1 a = 75
2
3 if a > 50 :
4     if a < 100 :
5         print("50보다 크고 100보다 작군요.")
6     else :
7         print("와~~ 100보다 크군요.")
8 else :
9     print("에고~ 50보다 작군요.")
```

출력 결과

50보다 크고 100보다 작군요.

2 제어문

■ if~elif~else 문

Code05-08.py

```

1  score = int(input("점수를 입력하세요 : "))
2
3  if score >= 90 :
4      print("A")
5  elif score >= 80 :
6      print("B")
7  elif score >= 70 :
8      print("C")
9  elif score >= 60 :
10     print("D")
11 else :
12     print("F")
13
14 print("학점입니다. ^^")

```

2 제어문

■ 리스트와 함께 사용

- 리스트(List) : 데이터 여러 개를 한곳에 담아 놓은 것
- 방법 : 대괄호 []로 묶고 그 안에 필요한 것들을 한꺼번에 넣음
- 예 : fruit 변수에 값 4개를 리스트 하나로 묶어 대입

```
fruit = ['사과', '배', '딸기', '포도']
print(fruit)
```

출력 결과

```
['사과', '배', '딸기', '포도']
```

- 추가

```
fruit.append('귤')
print(fruit)
```

출력 결과

```
['사과', '배', '딸기', '포도', '귤']
```

2 제어문

■ for 문의 개념

■ 기본 형식

```
for 변수 in range(시작값, 끝값+1, 증가값) :  
    이 부분을 반복
```

- 예 : range() 함수는 시퀀스 객체를 생성하며 for 함수에 주로 쓰임

```
for i in range(0, 3, 1) :  
    print("안녕하세요? for 문을 공부 중입니다. ^^")
```

```
for i in [0, 1, 2] :  
    print("안녕하세요? for 문을 공부 중입니다. ^^")
```



2 제어문

- range() 함수의 시작값 2, i값을 1씩 줄여(0이 될 때까지) print() 함수 3번 실행

```
for i in range(2, -1, -1) :
    print("%d : 안녕하세요? for 문을 공부 중입니다. ^^" % i)
```

출력 결과

```
2 : 안녕하세요? for 문을 공부 중입니다. ^^
1 : 안녕하세요? for 문을 공부 중입니다. ^^
0 : 안녕하세요? for 문을 공부 중입니다. ^^
```

- 1~5의 숫자들을 차례로 출력

```
for i in range(1, 6, 1) :
    print("%d " % i, end = " ")
```

출력 결과

```
1 2 3 4 5
```



2 제어문

■ 중첩 for 문의 개념

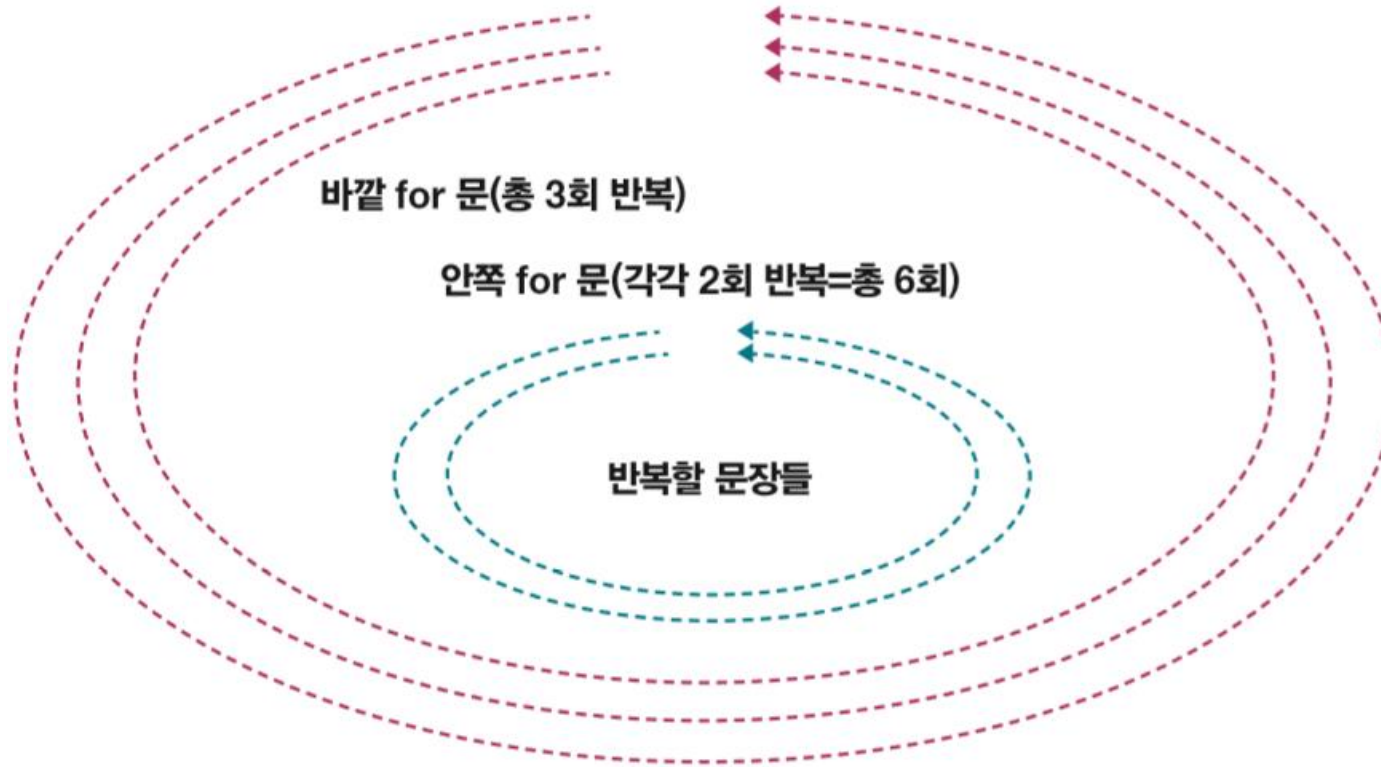


그림 6-2 중첩 for 문의 작동 개념

2 제어문

- 중첩 for 문의 기본 형식

```
for i in range(0, 3, 1) :
    for k in range(0, 2, 1) :
        print("파이썬은 꿀잼입니다. ^^ (i값 : %d, k값 : %d)" % (i, k))
```

출력 결과

```
파이썬은 꿀잼입니다. ^^ (i값 : 0, k값 : 0)
파이썬은 꿀잼입니다. ^^ (i값 : 0, k값 : 1)
파이썬은 꿀잼입니다. ^^ (i값 : 1, k값 : 0)
파이썬은 꿀잼입니다. ^^ (i값 : 1, k값 : 1)
파이썬은 꿀잼입니다. ^^ (i값 : 2, k값 : 0)
파이썬은 꿀잼입니다. ^^ (i값 : 2, k값 : 1)
```


2 제어문

- 중첩 for 문에서 i와 k값 변화

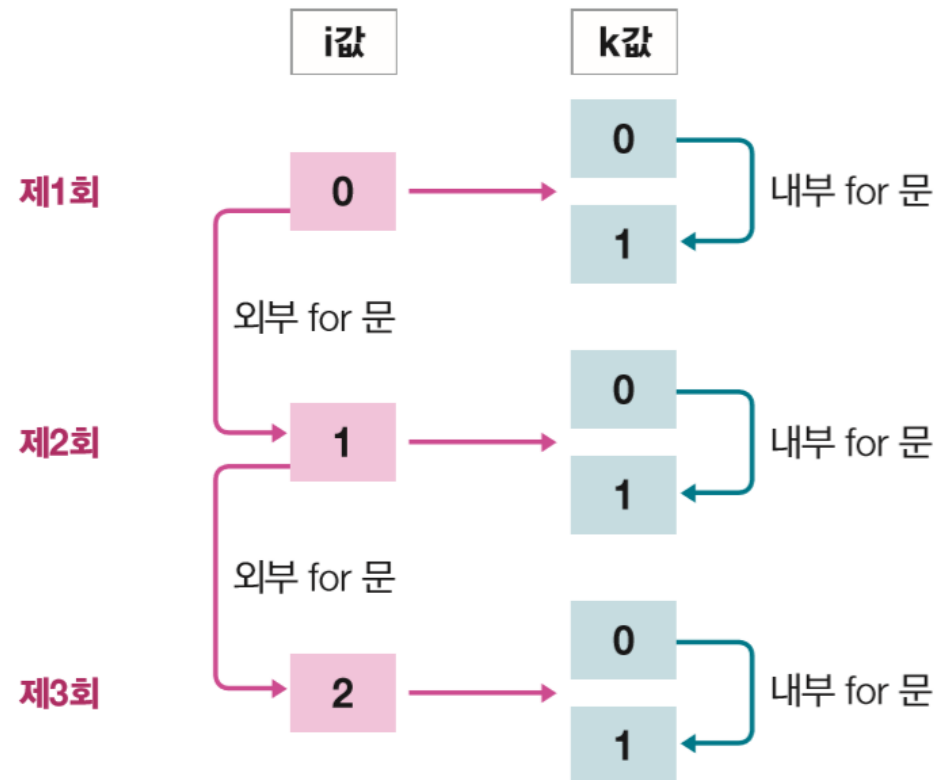


그림 6-3 중첩 for 문에서 i와 k값 변화

2 제어문

■ for 문과 while 문 비교

■ for 문의 형식

```
for 변수 in range(시작값, 끝값+1, 증가값)
```

- for 문은 반복할 횟수를 range() 함수에서 결정 후 그 횟수만큼 반복, while 문은 반복 횟수를 결정하기보다는 조건식이 참일 때 반복하는 방식

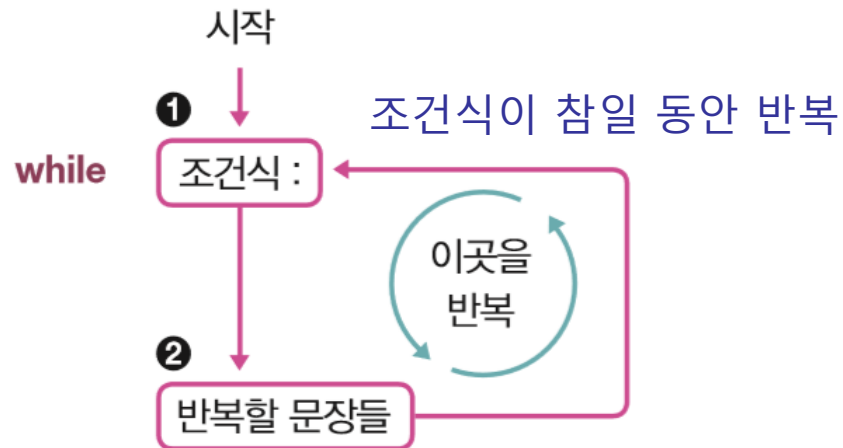


그림 6-6 while 문의 형식과 순서도

2 제어문

- for 문과 비슷하게 사용할 수 있는 while 문의 형식

```
변수 = 시작값
while 변수 < 끝값 :
    이 부분을 반복
    변수 = 변수 + 증가값
```

- for 문으로 '안녕하세요?~' 문장을 3회 출력하는 코드

```
for i in range(0, 3, 1) :
    print("%d : 안녕하세요? for 문을 공부 중입니다. ^^" % i)
```

2 제어문

- 문장을 3회 반복하도록 while 문

```
for i in range(0, 3, 1) :  
    print("%d : 안녕하세요? for 문을 공부 중입니다. ^^" % i)
```

```
i = 0  
while i < 3 :  
    print("%d : 안녕하세요? while 문을 공부 중입니다. ^^" % i)  
    i = i + 1
```

출력 결과

```
0 : 안녕하세요? while 문을 공부 중입니다. ^^  
1 : 안녕하세요? while 문을 공부 중입니다. ^^  
2 : 안녕하세요? while 문을 공부 중입니다. ^^
```

2 제어문

■ 무한 루프를 하는 while 문

- 무한 루프 적용 : 'while 조건식 :'에 들어가는 조건식을 True로 지정

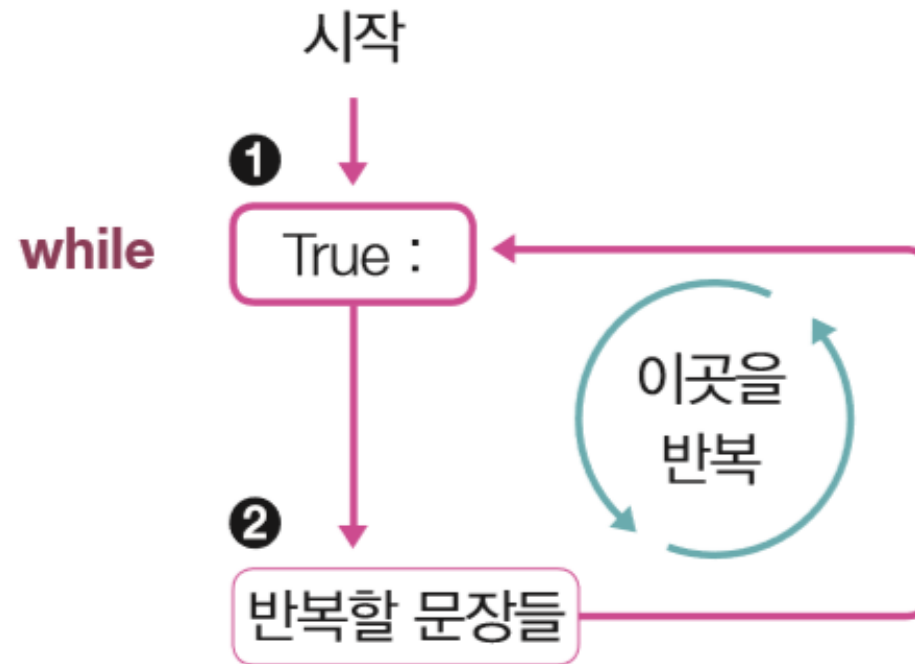


그림 6-7 while 문을 이용한 무한 루프

2 제어문

■ 반복문을 탈출에 **break** 문 사용

- 계속되는 반복을 논리적으로 빠져나가는 방법

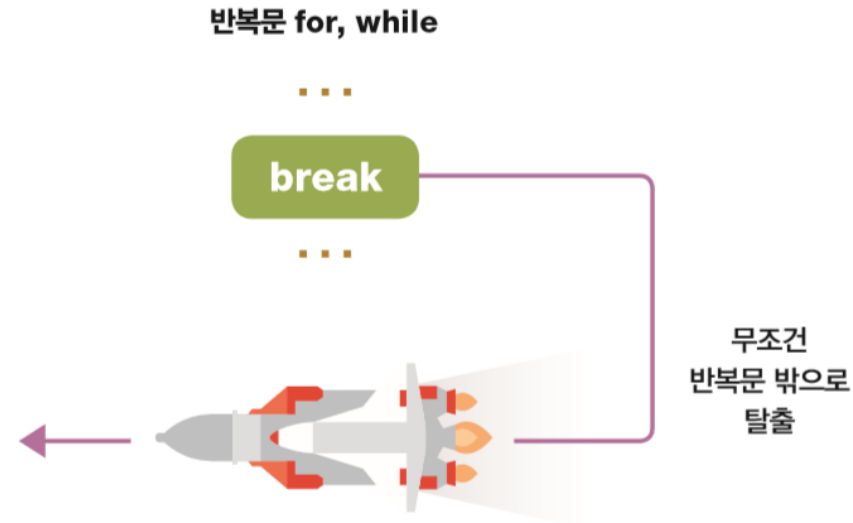


그림 6-8 break 문의 작동

```
for i in range(1, 100) :
    print("for 문을 %d번 실행했습니다." % i)
    break
```

출력 결과

for 문을 1번 실행했습니다.

2 제어문

- 반복문 loop의 시작점으로 다시 돌아가게 하는 **continue** 문

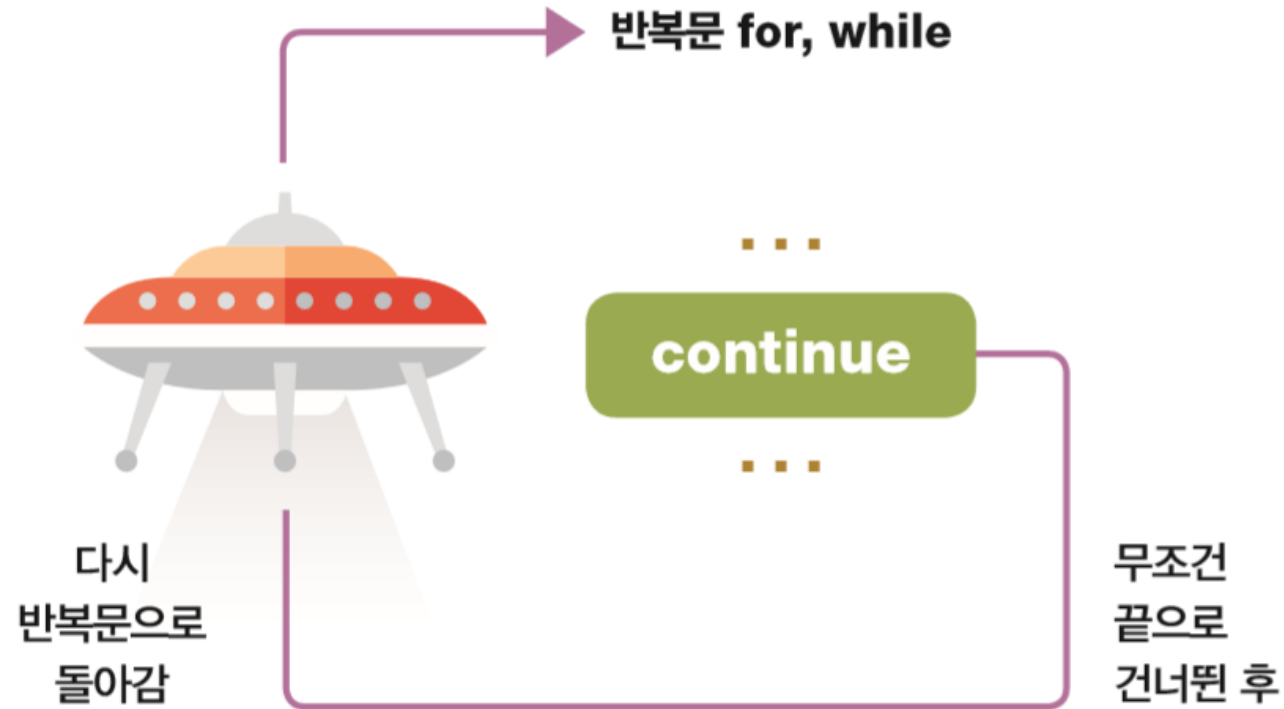


그림 6-9 continue 문의 작동

Q & A

